Dokumentacja projektu „Przepisy Kulinarne”

Małgorzata Pytlak, Julia Radosz

17 marca 2026

Spis treści

1 Wprowadzenie **4**
1.1 Cel projektu 4
1.2 Opis aplikacji 4
1.3 Zakres funkcjonalności 4

2 Wymagania projektowe **5**
2.1 Wymagania funkcjonalne 5
2.2 Wymagania niefunkcjonalne 6
2.3 Zgodność z wymaganiami projektu 6

3 Opis systemu **7**
3.1 Architektura aplikacji 7
3.2 Technologie wykorzystane w projekcie 7
3.3 Struktura projektu Django 8

4 Modele danych **8**
4.1 Opis modeli 8
4.1.1 Category 9
4.1.2 Recipe 9
4.1.3 Ingredient 9
4.1.4 Comment 10
4.1.5 UserProfile 10
4.1.6 Tag 10
4.2 Relacje między modelami 10

5 Funkcjonalności aplikacji **11**
5.1 Przeglądanie przepisów 11
5.2 Dodawanie przepisów 12
5.3 Edycja i usuwanie przepisów 13
5.4 Kategorie przepisów 14
5.5 Komentarze i oceny 15
5.6 System użytkowników 15
5.7 Wyszukiwanie przepisów 16
5.8 Ulubione przepisy 17

6 Widoki i routing **17**
6.1 Widoki aplikacji . . . 17
6.2 Routing . . . 18
6.3 Obsługa błędów . . . 18

7 Formularze **18**
7.1 Formularz dodawania przepisu . . . 18
7.2 Formularz komentarzy . . . 19
7.3 Walidacja danych . . . 19

8 Szablony i interfejs użytkownika **19**
8.1 Struktura szablonów . . . 19
8.2 Lista przepisów . . . 20
8.3 Szczegóły przepisu . . . 20
8.4 Kategorie . . . 20
8.5 Responsywność i stylizacja . . . 20

9 Komendy Django **21**
9.1 seed_data . . . 21
9.2 import_recipes . . . 21
9.3 import_ingredients . . . 21
9.4 delete_old_recipes . . . 21
9.5 download_images . . . 22

10 Import danych **22**
10.1 Import przepisów z pliku CSV . . . 22
10.2 Struktura pliku CSV . . . 22
10.3 Przetwarzanie danych . . . 22

11 Testowanie **23**
11.1 Testy modeli . . . 23
11.2 Testy widoków . . . 23
11.3 Testy formularzy . . . 24
11.4 Testy komend . . . 24
11.5 Podsumowanie testów . . . 24

12 Uruchomienie projektu **24**
12.1 Instalacja zależności . . . 25
12.2 Migracje bazy danych . . . 25
12.3 Uruchomienie serwera . . . 25

13 Podsumowanie **25**
13.1 Osiągnięte cele . . . 25
13.2 Możliwości rozwoju . . . 26

14 Podział obowiązków **26**
14.1 Julia Radosz . . . 26
14.2 Małgorzata Pytlak . . . 27

15 Źródła i wykorzystanie narzędzi **27**
15.1 Podział pracy: praca własna, dokumentacja i AI 27
15.2 Wykorzystane źródła internetowe 28
15.3 Wykorzystanie sztucznej inteligencji – źródła 29
15.4 Wykorzystane biblioteki i frameworki 29

1 Wprowadzenie

1.1 Cel projektu

Celem projektu było zaprojektowanie oraz implementacja aplikacji webowej umożliwiającej zarządzanie przepisami kulinarnymi. Aplikacja została wykonana z wykorzystaniem frameworka Django i spełnia wymagania określone w specyfikacji projektowej, takie jak obsługa wielu modeli danych, implementacja operacji CRUD oraz wykorzystanie formularzy i testów jednostkowych.

Głównym założeniem projektu było stworzenie intuicyjnego systemu, który pozwala użytkownikom na przeglądanie, dodawanie oraz ocenianie przepisów. Aplikacja umożliwia również organizację przepisów w kategorie, co ułatwia ich wyszukiwanie i przeglądanie.

Dodatkowym celem było zapoznanie się z architekturą aplikacji webowych w Django, pracą z bazą danych oraz implementacją własnych komend zarządzających (management commands).

1.2 Opis aplikacji

Aplikacja „Przepisy Kulinarne" jest systemem webowym umożliwiającym użytkownikom zarządzanie przepisami kulinarnymi. System pozwala na tworzenie, edycję oraz usuwanie przepisów, a także ich przeglądanie przez innych użytkowników.

Każdy przepis zawiera szczegółowe informacje, takie jak tytuł, opis, lista składników, instrukcja przygotowania, czas przygotowania oraz poziom trudności. Przepisy są przypisane do kategorii, co pozwala na ich uporządkowanie i łatwiejszą nawigację w aplikacji.

Aplikacja posiada system kont użytkowników, dzięki któremu możliwe jest przypisywanie przepisów do konkretnych autorów. Użytkownicy mogą również dodawać komentarze i oceny do przepisów, co zwiększa interaktywność systemu.

Dodatkowo zaimplementowano funkcję wyszukiwania przepisów oraz mechanizm oznaczania ulubionych dań. System umożliwia również import danych z plików CSV oraz automatyczne pobieranie zdjęć dla przepisów.

1.3 Zakres funkcjonalności

Zakres funkcjonalności aplikacji obejmuje:

- przeglądanie listy przepisów wraz z podstawowymi informacjami,
- wyświetlanie szczegółów pojedynczego przepisu,
- dodawanie nowych przepisów przez zalogowanych użytkowników,
- edycję oraz usuwanie istniejących przepisów,
- przypisywanie przepisów do kategorii,
- dodawanie składników do przepisów,
- system komentarzy i ocen (w skali od 1 do 5),
- rejestrację i logowanie użytkowników,
- tworzenie i zarządzanie profilem użytkownika,

- wyszukiwanie przepisów po różnych kryteriach,
- oznaczanie przepisów jako ulubione,
- import przepisów z plików CSV,
- automatyczne pobieranie zdjęć do przepisów,
- panel administracyjny umożliwiający zarządzanie danymi aplikacji.

2 Wymagania projektowe

2.1 Wymagania funkcjonalne

Wymagania funkcjonalne określają, jakie operacje i działania musi umożliwiać aplikacja. Do najważniejszych funkcjonalności systemu należą:

- przeglądanie listy przepisów dostępnych w systemie,
- wyświetlanie szczegółowych informacji o przepisie (opis, składniki, instrukcja, czas przygotowania, poziom trudności),
- dodawanie nowych przepisów przez zalogowanych użytkowników,
- edycja istniejących przepisów przez ich autorów,
- usuwanie przepisów,
- przypisywanie przepisów do kategorii,
- dodawanie i wyświetlanie składników przypisanych do przepisu,
- dodawanie komentarzy oraz ocen do przepisów,
- rejestracja nowych użytkowników oraz logowanie do systemu,
- zarządzanie profilem użytkownika,
- wyszukiwanie przepisów na podstawie różnych kryteriów (np. tytuł, opis),
- oznaczanie przepisów jako ulubione,
- import przepisów z pliku CSV za pomocą komendy zarządzającej,
- usuwanie starych przepisów przy użyciu dedykowanej komendy,
- automatyczne pobieranie zdjęć dla przepisów.

2.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają cechy jakościowe systemu oraz ograniczenia technologiczne.

- aplikacja została zaimplementowana w języku Python z wykorzystaniem frameworka Django,
- system korzysta z relacyjnej bazy danych SQLite,
- kod aplikacji jest zgodny z zasadami czytelności i dobrych praktyk programowania (m.in. PEP8),
- struktura projektu jest przejrzysta i logicznie podzielona na moduły (modele, widoki, formularze),
- interfejs użytkownika jest responsywny i dostosowany do różnych rozdzielczości ekranu (Bootstrap),
- dane wejściowe są walidowane po stronie formularzy,
- aplikacja obsługuje podstawowe błędy, takie jak brak strony (404),
- system zapewnia integralność danych dzięki wykorzystaniu relacji w bazie danych,
- operacje na danych są realizowane w sposób bezpieczny (np. uwierzytelnianie użytkowników),
- aplikacja umożliwia łatwe rozszerzanie o nowe funkcjonalności.

2.3 Zgodność z wymaganiami projektu

Projekt został zrealizowany zgodnie z wymaganiami określonymi w specyfikacji zadania.

Aplikacja spełnia następujące kryteria:

- zawiera więcej niż pięć podstron (m.in. lista przepisów, szczegóły przepisu, dodawanie przepisu, logowanie, profil użytkownika),
- implementuje operacje CRUD (Create, Read, Update, Delete) dla przepisów,
- wykorzystuje co najmniej pięć modeli danych powiązanych relacjami (Category, Recipe, Ingredient, Comment, UserProfile, Tag),
- zawiera formularze umożliwiające wprowadzanie danych (np. dodawanie przepisu, komentarze),
- posiada własne komendy Django (seed_data, import_recipes, delete_old_recipes, download_images),
- umożliwia komunikację z bazą danych w obu kierunkach,
- zawiera testy jednostkowe obejmujące kluczowe elementy aplikacji,
- wykorzystuje stylizację z użyciem frameworka Bootstrap,

- umożliwia rejestrację i logowanie użytkowników,
- zawiera system komentarzy,
- umożliwia wyszukiwanie przepisów.

Na podstawie powyższych elementów można stwierdzić, że projekt spełnia wszystkie wymagania funkcjonalne i techniczne określone dla zadania.

3 Opis systemu

3.1 Architektura aplikacji

Aplikacja została zaprojektowana w architekturze klient–serwer. Użytkownik korzysta z aplikacji za pomocą przeglądarki internetowej, natomiast logika biznesowa oraz obsługa danych realizowana jest po stronie serwera przy użyciu frameworka Django.

System wykorzystuje architekturę warstwową, w której można wyróżnić następujące elementy:

- **Warstwa prezentacji** – odpowiada za wyświetlanie danych użytkownikowi. Została zrealizowana przy użyciu szablonów HTML oraz stylów CSS (Bootstrap).
- **Warstwa logiki aplikacji** – obejmuje widoki Django (views), które przetwarzają żądania użytkownika, wykonują operacje na danych oraz zwracają odpowiednie odpowiedzi.
- **Warstwa danych** – reprezentowana przez modele Django, które odpowiadają za strukturę danych oraz komunikację z bazą danych.

Komunikacja między użytkownikiem a systemem odbywa się poprzez protokół HTTP. Użytkownik wysyła żądanie do serwera (np. otwarcie strony lub wysłanie formularza), a serwer przetwarza je i zwraca odpowiedź w postaci wygenerowanej strony HTML.

3.2 Technologie wykorzystane w projekcie

W projekcie wykorzystano następujące technologie:

- **Python** – główny język programowania użyty do implementacji aplikacji,
- **Django** – framework webowy umożliwiający szybkie tworzenie aplikacji internetowych,
- **SQLite** – relacyjna baza danych wykorzystywana do przechowywania danych aplikacji,
- **HTML** – język znaczników używany do budowy interfejsu użytkownika,
- **CSS** – język stylów wykorzystywany do wyglądu aplikacji,
- **Bootstrap 5** – framework CSS umożliwiający tworzenie responsywnego interfejsu,
- **Pillow** – biblioteka Python wykorzystywana do obsługi obrazów,

- **duckduckgo_search / ddgs** – biblioteka używana do automatycznego pobierania zdjęć dla przepisów.

Zastosowanie powyższych technologii pozwoliło na stworzenie kompletnej aplikacji webowej z interfejsem użytkownika, obsługą danych oraz dodatkowymi funkcjonalnościami.

3.3 Struktura projektu Django

Projekt został zorganizowany zgodnie z konwencją frameworka Django i składa się z głównego projektu oraz aplikacji.

Struktura projektu obejmuje:

- **przepisy_project**/ – główny katalog projektu zawierający konfigurację Django (pliki settings.py, urls.py),
- **recipes**/ – główna aplikacja projektu, zawierająca:
  - **models.py** – definicje modeli danych,
  - **views.py** – logika obsługi żądań użytkownika,
  - **forms.py** – definicje formularzy,
  - **urls.py** – konfiguracja tras (routing),
  - **admin.py** – konfiguracja panelu administracyjnego,
  - **management/commands**/ – własne komendy Django,
  - **tests**/ – testy jednostkowe aplikacji.
- **templates**/ – katalog zawierający szablony HTML wykorzystywane do renderowania widoków,
- **static**/ – pliki statyczne, takie jak arkusze stylów CSS,
- **media**/ – katalog przechowujący przesyłane przez użytkowników pliki (np. zdjęcia przepisów),
- **manage.py** – główny plik służący do zarządzania projektem (uruchamianie serwera, migracje, komendy).

Taka struktura projektu zapewnia przejrzystość kodu oraz ułatwia jego rozwój i utrzymanie.

4 Modele danych

4.1 Opis modeli

Aplikacja wykorzystuje relacyjną bazę danych, w której dane przechowywane są w postaci powiązanych ze sobą modeli. Modele zostały zaimplementowane przy użyciu ORM (Object-Relational Mapping) dostępnego w frameworku Django.

Każdy model reprezentuje określony typ danych w systemie oraz odpowiada tabeli w bazie danych.

4.1.1 Category

Model **Category** reprezentuje kategorię, do której przypisane są przepisy (np. „Zupy”, „Desery”, „Dania główne”).

Model zawiera podstawowe informacje, takie jak:

- nazwa kategorii,
- opis kategorii,
- opcjonalne zdjęcie kategorii.

Kategorie umożliwiają uporządkowanie przepisów oraz ułatwiają ich przeglądanie przez użytkownika.

4.1.2 Recipe

Model **Recipe** jest centralnym elementem aplikacji i reprezentuje pojedynczy przepis kulinarny.

Zawiera następujące informacje:

- tytuł przepisu,
- opis,
- instrukcję przygotowania,
- czas przygotowania i gotowania,
- liczbę porcji,
- poziom trudności,
- zdjęcie przepisu,
- datę utworzenia,
- autora przepisu.

Model ten odpowiada za przechowywanie wszystkich kluczowych danych związanych z przepisami.

4.1.3 Ingredient

Model **Ingredient** reprezentuje składniki przypisane do konkretnego przepisu.

Każdy składnik zawiera:

- nazwę składnika,
- ilość,
- jednostkę miary,
- powiązanie z przepisem.

Dzięki temu modelowi możliwe jest przechowywanie listy składników dla każdego przepisu.

4.1.4 Comment

Model **Comment** odpowiada za system komentarzy i ocen przepisów.

Zawiera następujące dane:

- treść komentarza,
- ocenę (np. w skali 1–5),
- datę dodania,
- autora komentarza,
- powiązanie z przepisem.

Model ten umożliwia użytkownikom dzielenie się opiniami na temat przepisów.

4.1.5 UserProfile

Model **UserProfile** rozszerza standardowy model użytkownika Django.

Zawiera dodatkowe informacje, takie jak:

- zdjęcie profilowe (avatar),
- dodatkowe dane użytkownika (np. opis profilu).

Model ten jest powiązany z modelem użytkownika relacją jeden-do-jednego.

4.1.6 Tag

Model **Tag** reprezentuje dodatkowe oznaczenia przepisów, takie jak „wegetariańskie", „szybkie", „fit".

Zawiera:

- nazwę tagu.

Tagi pozwalają na lepszą organizację oraz filtrowanie przepisów.

4.2 Relacje między modelami

W systemie zastosowano różne typy relacji między modelami, które umożliwiają powiązanie danych:

- **Category** – **Recipe** : relacja jeden-do-wielu (ForeignKey) Jedna kategoria może zawierać wiele przepisów, natomiast każdy przepis należy do jednej kategorii.
- **Recipe** – **Ingredient** : relacja jeden-do-wielu Jeden przepis może posiadać wiele składników, a każdy składnik należy do jednego przepisu.
- **Recipe** – **Comment** : relacja jeden-do-wielu Jeden przepis może mieć wiele komentarzy.
- **User** – **Recipe** : relacja jeden-do-wielu Jeden użytkownik może dodać wiele przepisów.

- **User – Comment** : relacja jeden-do-wielu Jeden użytkownik może dodać wiele komentarzy.
- **User – UserProfile** : relacja jeden-do-jednego (OneToOne) Każdy użytkownik posiada jeden profil.
- **Recipe – Tag** : relacja wiele-do-wielu (ManyToMany) Jeden przepis może mieć wiele tagów, a jeden tag może być przypisany do wielu przepisów.

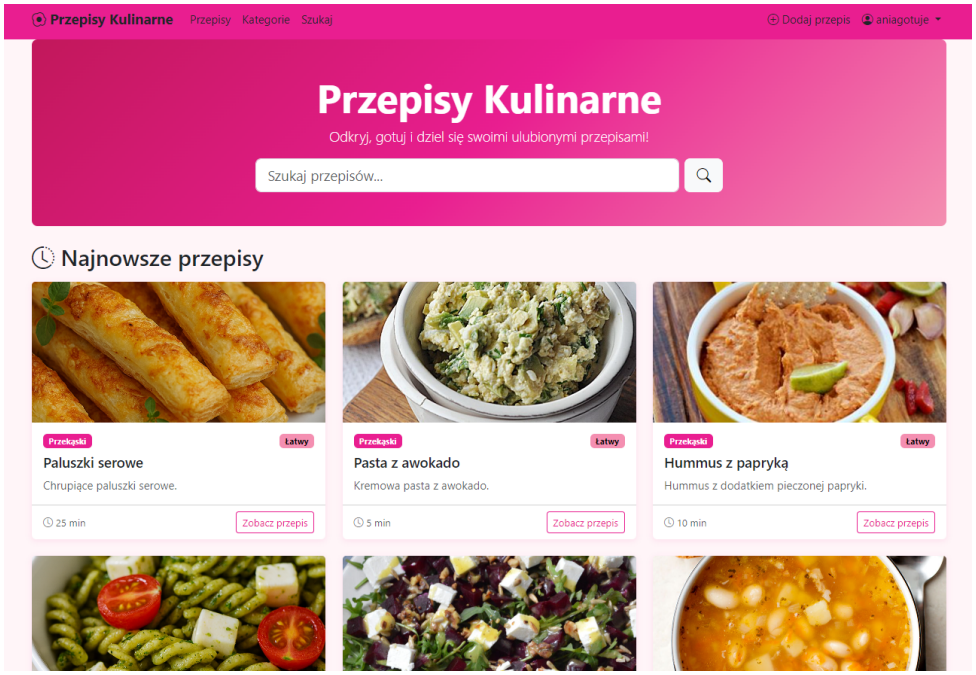
Zastosowanie powyższych relacji pozwala na spójne i efektywne zarządzanie danymi w aplikacji.

5 Funkcjonalności aplikacji

5.1 Przeglądanie przepisów

Aplikacja umożliwia użytkownikom przeglądanie listy dostępnych przepisów. Na stronie głównej wyświetlane są podstawowe informacje o przepisach, takie jak tytuł, zdjęcie oraz krótki opis.

Użytkownik może przejść do szczegółowego widoku przepisu, gdzie dostępne są wszystkie informacje, w tym składniki oraz instrukcja przygotowania.

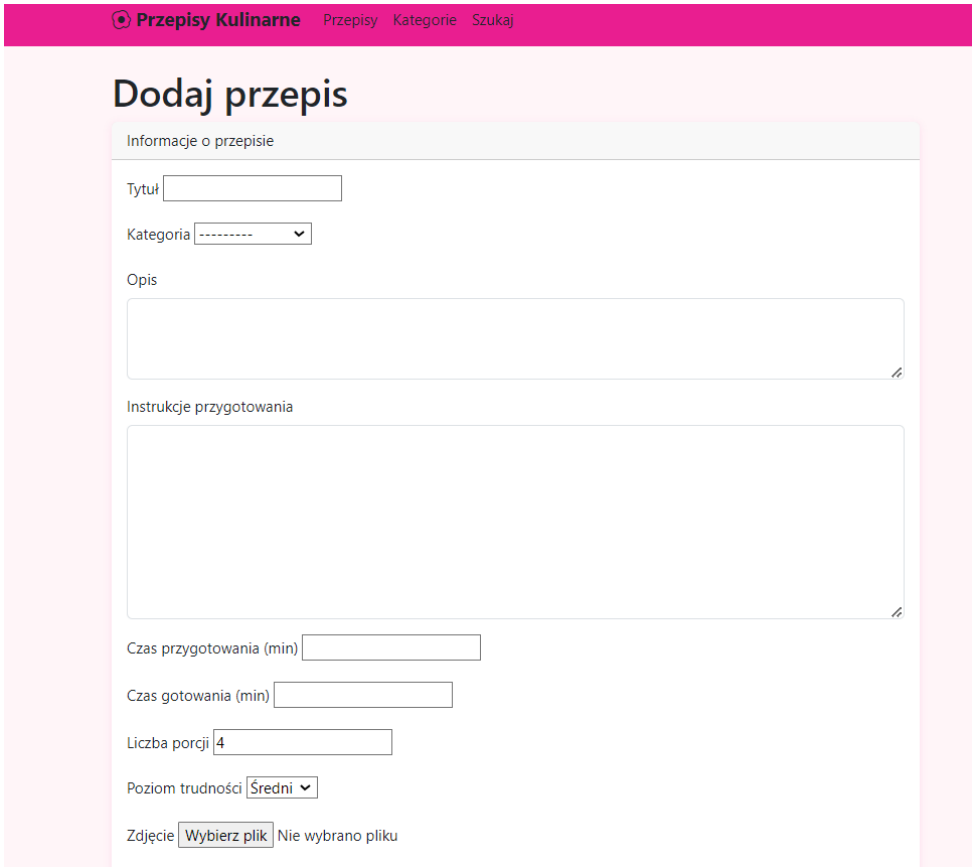


Rysunek 1: Widok strony głównej

5.2 Dodawanie przepisów

Zalogowani użytkownicy mają możliwość dodawania nowych przepisów za pomocą formularza. Formularz umożliwia wprowadzenie wszystkich niezbędnych danych, takich jak tytuł, opis, składniki, instrukcja oraz zdjęcie.

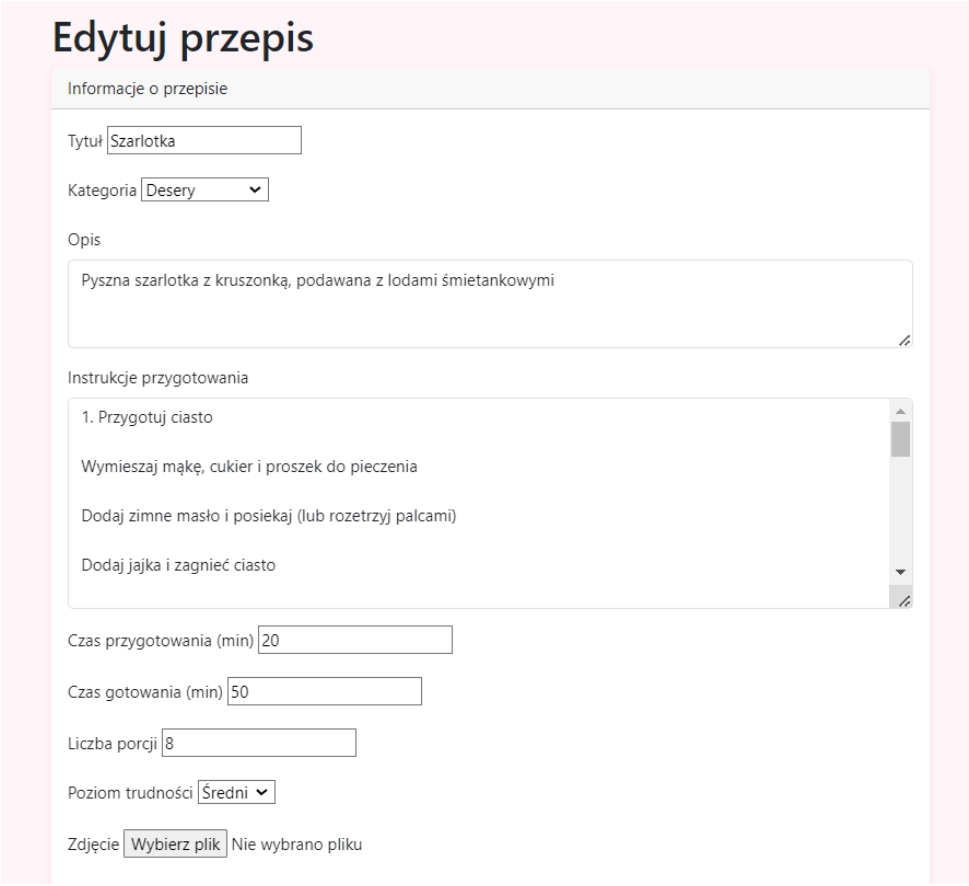
Dane są walidowane przed zapisaniem w bazie danych.



Rysunek 2: Formularz dodawania przepisu

5.3 Edycja i usuwanie przepisów

Autor przepisu ma możliwość jego edycji oraz usunięcia. Edycja odbywa się poprzez formularz podobny do formularza dodawania, natomiast usunięcie przepisu powoduje trwałe usunięcie danych z bazy.

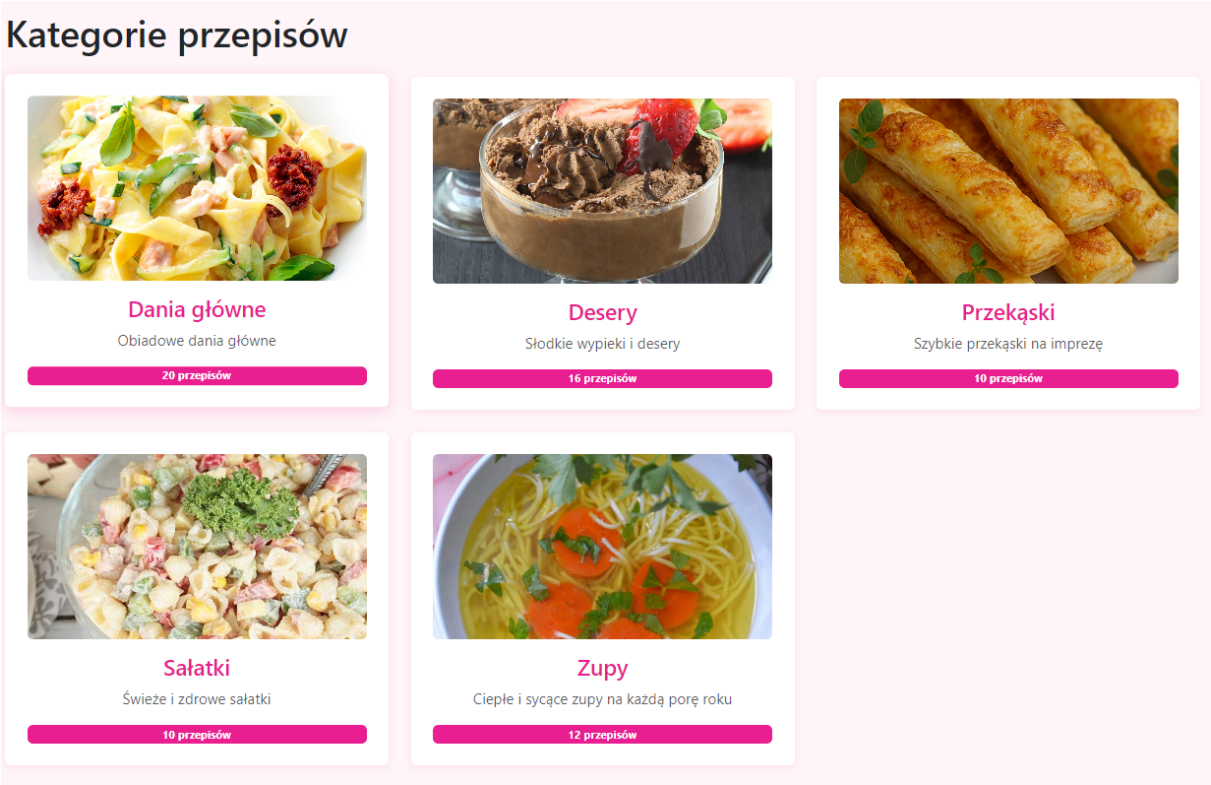


Rysunek 3: Widok edycji przepisu

5.4 Kategorie przepisów

Przepisy są przypisane do kategorii, co umożliwia ich uporządkowanie. Użytkownik może przeglądać przepisy w obrębie wybranej kategorii.

Kategorie mogą zawierać również własne opisy oraz zdjęcia.



Rysunek 4: Widok kategorii przepisów

5.5 Komentarze i oceny

Użytkownicy mogą dodawać komentarze oraz oceny do przepisów. System ocen pozwala na wystawienie oceny w określonej skali, co umożliwia innym użytkownikom ocenę jakości przepisu.

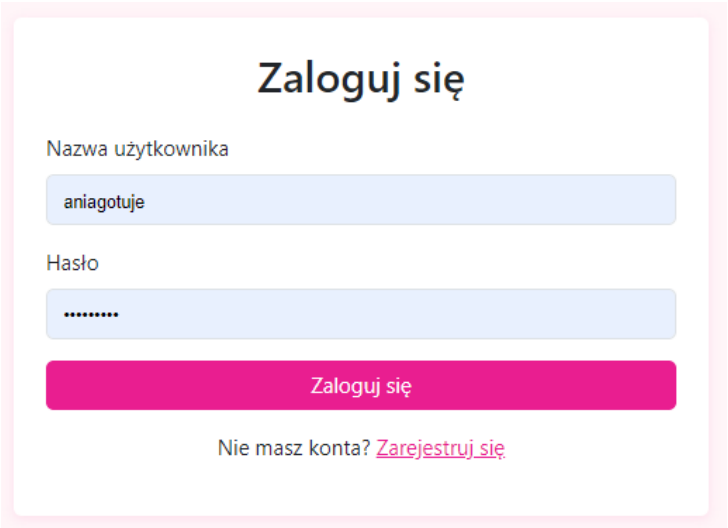
Komentarze są wyświetlane pod przepisem.

Rysunek 5: Sekcja komentarzy pod przepisem

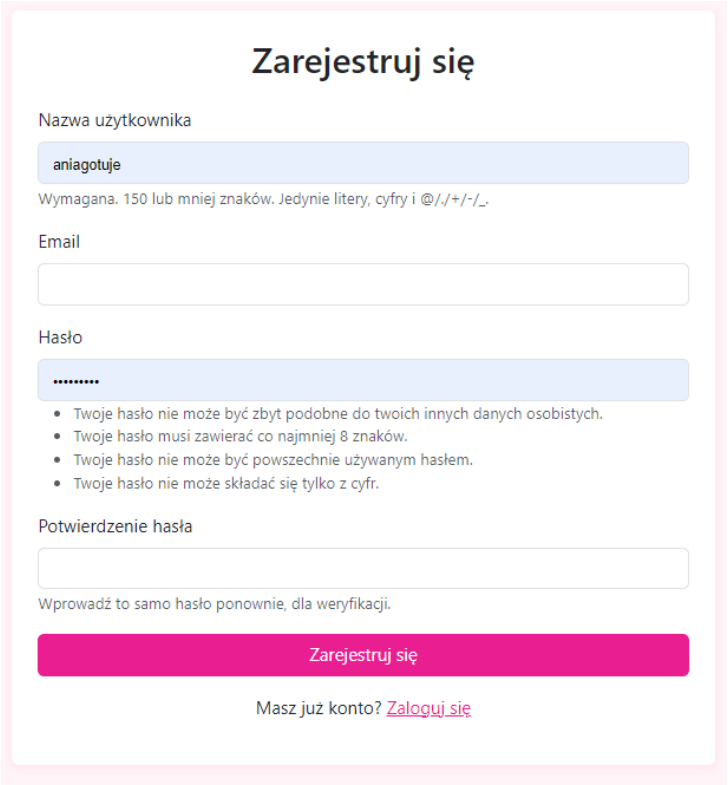
5.6 System użytkowników

Aplikacja posiada system rejestracji i logowania użytkowników. Każdy użytkownik posiada własny profil oraz możliwość zarządzania swoimi przepisami.

Dostęp do niektórych funkcji (np. dodawanie przepisów) wymaga zalogowania.



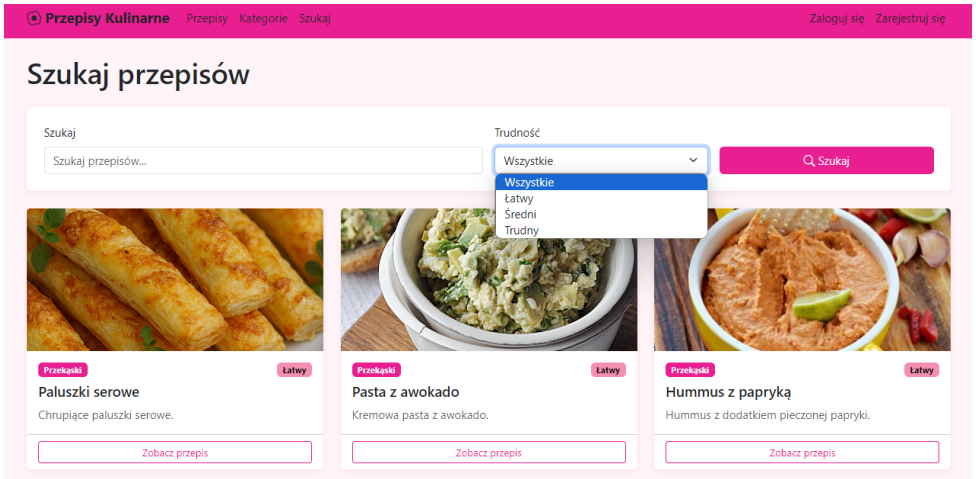
Rysunek 6: Widok logowania użytkownika



Rysunek 7: Widok rejestracji użytkownika

5.7 Wyszukiwanie przepisów

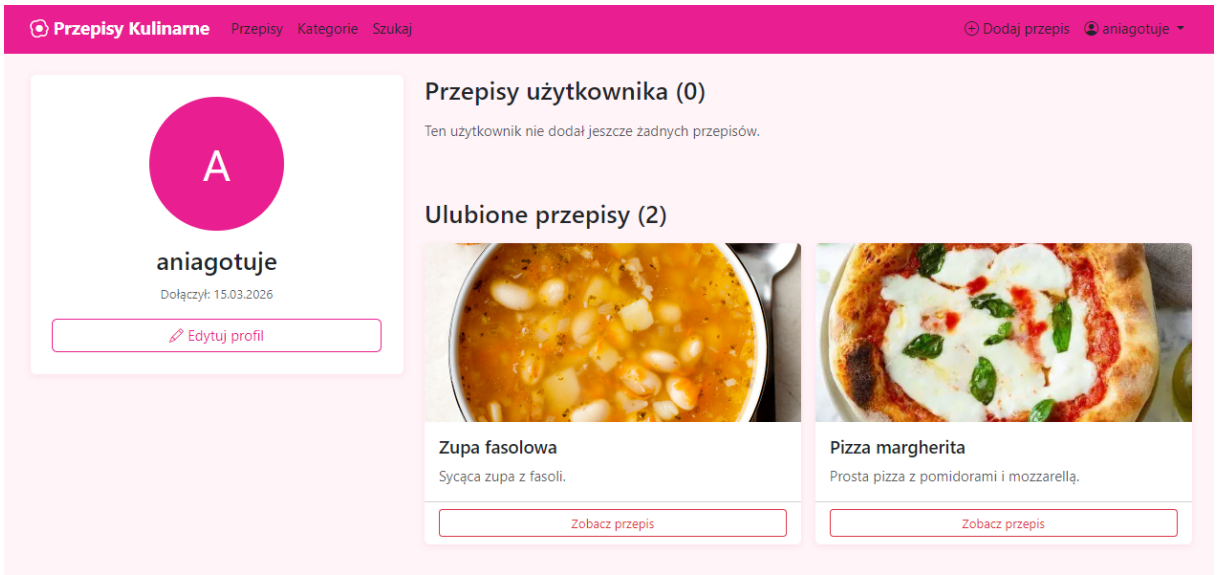
System umożliwia wyszukiwanie przepisów na podstawie różnych kryteriów, takich jak tytuł lub opis. Funkcja wyszukiwania pozwala użytkownikowi szybko znaleźć interesujące go przepisy.



Rysunek 8: Widok wyszukiwania przepisów

5.8 Ulubione przepisy

Zalogowani użytkownicy mogą oznaczać przepisy jako ulubione. Dzięki temu mają szybki dostęp do zapisanych przepisów w swoim profilu.



Rysunek 9: Widok ulubionych przepisów

6 Widoki i routing

6.1 Widoki aplikacji

Widoki (ang. *views*) w aplikacji Django odpowiadają za obsługę żądań użytkownika oraz zwracanie odpowiednich odpowiedzi w postaci stron HTML.

W projekcie zastosowano widoki oparte na funkcjach (function-based views), które realizują logikę aplikacji. Każdy widok odpowiada za konkretną funkcjonalność, taką jak wyświetlanie listy przepisów, szczegółów przepisu czy obsługa formularzy.

Widoki pobierają dane z bazy danych przy użyciu modeli, a następnie przekazują je do szablonów, które generują interfejs użytkownika.

Przykładowe widoki w aplikacji:

- widok listy przepisów,
- widok szczegółów przepisu,
- widok dodawania i edycji przepisu,
- widok kategorii,
- widok logowania i rejestracji użytkownika.

Dzięki zastosowaniu widoków możliwe jest oddzielenie logiki aplikacji od warstwy prezentacji.

6.2 Routing

Routing w Django odpowiada za mapowanie adresów URL na odpowiednie widoki.

W projekcie wykorzystano pliki `urls.py`, w których zdefiniowane są ścieżki prowadzące do konkretnych funkcji widoków. Każdy adres URL odpowiada określonej funkcjonalności aplikacji.

Przykładowe trasy w aplikacji:

- `/` – strona główna z listą przepisów,
- `/recipe/<slug>/` – szczegóły przepisu,
- `/add/` – dodawanie nowego przepisu,
- `/category/<slug>/` – lista przepisów w danej kategorii,
- `/login/` – logowanie użytkownika.

Routing umożliwia przejrzyste zarządzanie nawigacją w aplikacji oraz oddzielenie logiki URL od implementacji widoków.

6.3 Obsługa błędów

Aplikacja obsługuje podstawowe błędy, które mogą wystąpić podczas korzystania z systemu.

W szczególności zaimplementowano obsługę błędu 404 (strona nie została znaleziona). W przypadku próby wejścia na nieistniejący adres URL użytkownikowi wyświetlany jest odpowiedni komunikat.

Dodatkowo system może obsługiwać inne błędy, takie jak błędy formularzy czy brak dostępu do zasobów, informując użytkownika o problemie w czytelny sposób.

Obsługa błędów zwiększa użyteczność aplikacji oraz poprawia doświadczenie użytkownika.

7 Formularze

7.1 Formularz dodawania przepisu

W aplikacji wykorzystano formularze Django do wprowadzania danych przez użytkownika. Formularz dodawania przepisu umożliwia wprowadzenie wszystkich niezbędnych informacji wymaganych do utworzenia nowego przepisu.

Formularz zawiera pola takie jak:

- tytuł przepisu,
- opis,
- instrukcja przygotowania,
- czas przygotowania i gotowania,
- liczba porcji,
- poziom trudności,

- wybór kategorii,
- dodanie zdjęcia.

Po wypełnieniu formularza dane są przesyłane do serwera i zapisywane w bazie danych. Formularz jest dostępny wyłącznie dla zalogowanych użytkowników.

7.2 Formularz komentarzy

Aplikacja umożliwia użytkownikom dodawanie komentarzy do przepisów za pomocą dedykowanego formularza.

Formularz komentarza zawiera:

- pole tekstowe na treść komentarza,
- pole wyboru oceny (np. w skali od 1 do 5).

Po wysłaniu formularza komentarz zostaje zapisany w bazie danych i przypisany do konkretnego przepisu oraz użytkownika. Komentarze są następnie wyświetlane na stronie szczegółów przepisu.

7.3 Walidacja danych

W celu zapewnienia poprawności danych wprowadzanych przez użytkowników zastosowano mechanizmy walidacji formularzy.

Walidacja obejmuje:

- sprawdzenie, czy wymagane pola zostały wypełnione,
- kontrolę długości tekstu (np. tytułu i opisu),
- sprawdzenie poprawności wartości liczbowych (np. czas przygotowania, liczba porcji),
- weryfikację poprawności wybranych opcji (np. poziomu trudności),
- walidację przesyłanych plików (np. zdjęć).

W przypadku wprowadzenia niepoprawnych danych użytkownik otrzymuje odpowiedni komunikat błędu, a formularz nie zostaje zapisany.

Zastosowanie walidacji zwiększa bezpieczeństwo aplikacji oraz zapewnia spójność danych przechowywanych w systemie.

8 Szablony i interfejs użytkownika

8.1 Struktura szablonów

Interfejs użytkownika został zaimplementowany przy użyciu systemu szablonów Django. Szablony HTML odpowiadają za prezentację danych oraz komunikację z użytkownikiem.

Projekt wykorzystuje mechanizm dziedziczenia szablonów, co pozwala na uniknięcie powielania kodu. Główny szablon (np. `base.html`) definiuje wspólne elementy interfejsu, takie jak nagłówek, menu nawigacyjne oraz stopka. Pozostałe szablony rozszerzają go, definiując własną zawartość w odpowiednich blokach.

Takie podejście zapewnia spójność wizualną całej aplikacji oraz ułatwia jej rozwój.

8.2 Lista przepisów

Widok listy przepisów stanowi główną stronę aplikacji. Wyświetlane są na niej wszystkie dostępne przepisy w formie uporządkowanej listy lub siatki.

Każdy element listy zawiera podstawowe informacje, takie jak tytuł przepisu, zdjęcie oraz krótki opis. Użytkownik może kliknąć wybrany przepis, aby przejść do jego szczegółów.

Widok ten pełni funkcję punktu startowego dla użytkownika oraz umożliwia szybkie przeglądanie dostępnych treści.

8.3 Szczegóły przepisu

Strona szczegółów przepisu prezentuje wszystkie informacje związane z wybranym przepisem.

Na stronie wyświetlane są:

- tytuł przepisu,
- zdjęcie,
- opis,
- lista składników,
- instrukcja przygotowania,
- dodatkowe informacje (np. czas przygotowania, poziom trudności),
- komentarze użytkowników.

Widok ten umożliwia również interakcję z przepisem, np. dodanie komentarza lub oceny.

8.4 Kategorie

Aplikacja umożliwia przeglądanie przepisów według kategorii. Każda kategoria posiada własną stronę, na której wyświetlane są przypisane do niej przepisy.

Kategorie ułatwiają organizację danych oraz pozwalają użytkownikowi szybciej znaleźć interesujące go przepisy. Mogą być również przedstawione w formie listy lub kart zawierających nazwę oraz opis kategorii.

8.5 Responsywność i stylizacja

Interfejs użytkownika został zaprojektowany z wykorzystaniem frameworka Bootstrap 5, co pozwala na tworzenie responsywnych i estetycznych stron internetowych.

Responsywność oznacza, że aplikacja dostosowuje swój wygląd do różnych rozmiarów ekranu, takich jak komputery, tablety czy smartfony. Elementy interfejsu, takie jak siatki, przyciski oraz formularze, automatycznie zmieniają układ w zależności od dostępnej przestrzeni.

Zastosowanie Bootstrapa pozwoliło na szybkie stworzenie spójnego i czytelnego interfejsu użytkownika bez konieczności tworzenia wszystkich stylów od podstaw.

9 Komendy Django

W projekcie zaimplementowano własne komendy Django, które rozszerzają funkcjonalność aplikacji i umożliwiają automatyzację wybranych operacji na danych.

9.1 seed_data

Komenda `seed_data` służy do wypełnienia bazy danych przykładowymi danymi, takimi jak kategorie oraz przepisy.

```
class Command(BaseCommand):
    help = "Wype nia␣baz ␣przyk adowymi␣danymi"

    def handle(self, *args, **kwargs):
        Category.objects.get_or_create(name="Zupy")
        Category.objects.get_or_create(name="Desery")
        self.stdout.write("Dodano␣przyk adowe␣dane")
```

9.2 import_recipes

Komenda `import_recipes` umożliwia import przepisów z pliku CSV.

```
def handle(self, *args, **options):
    with open(options['csv_file'], encoding='utf-8') as file:
        reader = csv.DictReader(file)

        for row in reader:
            Recipe.objects.create(
                title=row['title'],
                description=row['description'],
                instructions=row['instructions']
            )
```

9.3 import_ingredients

Komenda `import_ingredients` pozwala na import składników przypisanych do przepisów.

```
for row in reader:
    recipe = Recipe.objects.get(title=row['recipe'])
    Ingredient.objects.create(
        recipe=recipe,
        name=row['name'],
        quantity=row['quantity']
    )
```

9.4 delete_old_recipes

Komenda `delete_old_recipes` usuwa stare przepisy na podstawie podanej liczby dni.

```
days = options['days']
date_limit = timezone.now() - timedelta(days=days)

Recipe.objects.filter(created_at__lt=date_limit).delete()
```

9.5 download_images

Komenda `download_images` automatycznie pobiera zdjęcia dla przepisów z internetu.

```
for recipe in Recipe.objects.all():
    query = recipe.title + "␣food"
    results = ddgs.images(query, max_results=1)

    for r in results:
        response = requests.get(r["image"])
        if response.status_code == 200:
            recipe.image.save(
                recipe.slug + ".jpg",
                ContentFile(response.content),
                save=True
            )
```

10 Import danych

10.1 Import przepisów z pliku CSV

Import danych realizowany jest przy użyciu komendy `import_recipes`. Umożliwia ona szybkie wprowadzenie dużej liczby przepisów bez konieczności ręcznego dodawania.

10.2 Struktura pliku CSV

Plik CSV wykorzystywany do importu danych posiada strukturę:

```
title,category,description,instructions,prep_time,cook_time,
   servings,difficulty
```

Każdy wiersz pliku odpowiada jednemu przepisowi.

10.3 Przetwarzanie danych

Podczas importu dane są odczytywane przy użyciu modułu `csv` i zapisywane w bazie danych.

Proces obejmuje:

- odczyt danych z pliku,
- przypisanie wartości do pól modelu,
- zapis obiektów w bazie danych,

- obsługę błędów (np. brak wymaganych danych).

Dzięki temu możliwe jest szybkie i efektywne zarządzanie dużą liczbą przepisów.

11 Testowanie

W projekcie zaimplementowano testy jednostkowe, których celem było sprawdzenie poprawności działania kluczowych elementów aplikacji. Testy zostały wykonane przy użyciu wbudowanego mechanizmu testowania frameworka Django.

Testy obejmują modele, widoki, formularze oraz własne komendy Django, co pozwala na kompleksową weryfikację działania systemu.

11.1 Testy modeli

Testy modeli mają na celu sprawdzenie poprawności działania logiki związanej z bazą danych.

W ramach testów modeli weryfikowane są:

- poprawność tworzenia obiektów,
- działanie relacji między modelami,
- unikalność wybranych pól (np. slug),
- poprawność metod modeli.

Testy te zapewniają, że dane są poprawnie zapisywane i odczytywane z bazy danych oraz że struktura modeli działa zgodnie z założeniami.

11.2 Testy widoków

Testy widoków sprawdzają poprawność działania logiki aplikacji oraz odpowiedzi serwera na żądania użytkownika.

W ramach testów weryfikowane są:

- poprawne działanie stron (status HTTP 200),
- poprawność przekazywanych danych do szablonów,
- działanie przekierowań (np. po dodaniu przepisu),
- dostęp do widoków wymagających logowania.

Testy widoków pozwalają upewnić się, że użytkownik otrzymuje poprawne odpowiedzi oraz że aplikacja działa zgodnie z oczekiwaniami.

11.3 Testy formularzy

Testy formularzy służą do sprawdzenia poprawności walidacji danych wprowadzanych przez użytkownika.

W testach sprawdzane są:

- poprawność danych wejściowych (formularz poprawny),
- wykrywanie błędnych danych (formularz niepoprawny),
- wymagane pola formularza,
- poprawność zakresów wartości (np. liczby porcji, czas przygotowania).

Dzięki tym testom możliwe jest wykrycie błędów związanych z wprowadzaniem danych oraz zapewnienie spójności informacji w systemie.

11.4 Testy komend

Testy komend Django weryfikują poprawność działania własnych poleceń zarządzających aplikacją.

Testowane są między innymi:

- poprawne wykonanie komendy bez błędów,
- wpływ komendy na bazę danych (np. dodanie lub usunięcie danych),
- obsługa argumentów przekazywanych do komend,
- poprawność komunikatów wyświetlanych w konsoli.

Testy te pozwalają upewnić się, że komendy działają zgodnie z założeniami i mogą być bezpiecznie wykorzystywane w praktyce.

11.5 Podsumowanie testów

Łącznie w projekcie zaimplementowano około 60 testów jednostkowych obejmujących różne warstwy aplikacji.

Testy pozwoliły na:

- wykrycie błędów w logice aplikacji,
- weryfikację poprawności działania modeli i relacji,
- sprawdzenie działania formularzy i walidacji danych,
- potwierdzenie poprawności działania widoków oraz komend Django.

Zastosowanie testów znacząco zwiększa jakość aplikacji oraz ułatwia jej dalszy rozwój i utrzymanie.

12 Uruchomienie projektu

Poniżej przedstawiono kroki niezbędne do uruchomienia aplikacji lokalnie.

12.1 Instalacja zależności

W pierwszym kroku należy zainstalować wszystkie wymagane biblioteki. Zaleca się użycie środowiska wirtualnego.

```
pip install -r requirements.txt
```

Projekt wykorzystuje między innymi framework Django oraz bibliotekę Pillow do obsługi obrazów.

12.2 Migracje bazy danych

Po zainstalowaniu zależności należy przygotować bazę danych poprzez wykonanie migracji:

```
python manage.py migrate
```

Migracje tworzą wszystkie wymagane tabele w bazie danych zgodnie z definicjami modeli.

Opcjonalnie można utworzyć konto administratora:

```
python manage.py createsuperuser
```

12.3 Uruchomienie serwera

Aby uruchomić aplikację lokalnie, należy wykonać polecenie:

```
python manage.py runserver
```

Po uruchomieniu aplikacja będzie dostępna pod adresem:

```
http://127.0.0.1:8000
```

Panel administracyjny dostępny jest pod adresem:

```
http://127.0.0.1:8000/admin
```

13 Podsumowanie

13.1 Osiągnięte cele

W ramach projektu zrealizowano aplikację webową umożliwiającą zarządzanie przepisami kulinarnymi.

Osiągnięto następujące cele:

- implementację systemu użytkowników (rejestracja, logowanie, profile),
- możliwość tworzenia, edycji i usuwania przepisów,
- wprowadzenie systemu komentarzy i ocen,
- implementację kategorii i wyszukiwania przepisów,

- dodanie funkcjonalności ulubionych przepisów,
- przygotowanie komend do importu danych i zarządzania bazą,
- stworzenie testów jednostkowych obejmujących kluczowe elementy aplikacji.

Aplikacja spełnia założenia projektowe i stanowi kompletny system do zarządzania przepisami.

13.2 Możliwości rozwoju

Projekt może być dalej rozwijany poprzez dodanie nowych funkcjonalności oraz ulepszenie istniejących rozwiązań.

Możliwe kierunki rozwoju:

- dodanie systemu polubień i ocen bez komentarzy,
- rozbudowa profili użytkowników (np. obserwowanie innych użytkowników),
- dodanie filtrowania przepisów według większej liczby kryteriów,
- integracja z zewnętrznymi API (np. do pobierania przepisów lub zdjęć),
- wdrożenie aplikacji na serwer produkcyjny,
- poprawa wydajności i optymalizacja zapytań do bazy danych.

Dalszy rozwój aplikacji pozwoliłby przekształcić ją w pełnoprawny serwis kulinarny dostępny dla szerokiego grona użytkowników.

14 Podział obowiązków

Projekt został zrealizowany zespołowo, a zakres prac został podzielony pomiędzy dwie osoby w następujący sposób:

14.1 Julia Radosz

Zakres pracy:

- implementacja modeli danych (Category, Recipe, Ingredient, Comment, UserProfile),
- implementacja widoków aplikacji (views.py),
- obsługa operacji CRUD dla przepisów,
- implementacja systemu komentarzy i ocen,
- przygotowanie formularzy Django,
- integracja aplikacji z bazą danych.

14.2 Małgorzata Pytlak

Zakres pracy:

- przygotowanie szablonów HTML oraz stylizacji (Bootstrap),
- implementacja profili użytkowników oraz systemu ulubionych przepisów,
- implementacja wyszukiwarki przepisów,
- przygotowanie własnych komend Django (manage.py),
- import danych z plików CSV,
- implementacja testów jednostkowych,
- przygotowanie dokumentacji projektu (README).

15 Źródła i wykorzystanie narzędzi

15.1 Podział pracy: praca własna, dokumentacja i AI

W trakcie realizacji projektu zastosowano podejście polegające na samodzielnej implementacji większości funkcjonalności, przy jednoczesnym wykorzystaniu dokumentacji, samouczków oraz narzędzi sztucznej inteligencji jako wsparcia w procesie tworzenia aplikacji.

Praca własna (większość projektu):

Znacząca część projektu została wykonana samodzielnie, w szczególności:

- zaprojektowanie architektury aplikacji Django (podział na modele, widoki, formularze),
- implementacja modeli danych oraz relacji między nimi,
- implementacja operacji CRUD dla przepisów,
- stworzenie systemu komentarzy i ocen,
- implementacja systemu ulubionych przepisów,
- stworzenie wyszukiwarki przepisów,
- przygotowanie szablonów HTML oraz integracja z Bootstrap,
- implementacja testów jednostkowych,
- stworzenie własnych komend Django,
- przygotowanie dokumentacji projektu.

Elementy te wymagały samodzielnego zaprojektowania logiki aplikacji, znajomości frameworka Django oraz umiejętności pracy z bazą danych.

Wykorzystanie dokumentacji i samouczków:

Dokumentacja i materiały edukacyjne były wykorzystywane głównie w celu poznania technologii oraz poprawnego użycia narzędzi:

- konfiguracja projektu Django i struktury aplikacji,
- wykorzystanie ORM Django (zapytania, relacje),
- obsługa formularzy Django (ModelForm, walidacja),
- system uwierzytelniania użytkowników,
- konfiguracja panelu administratora,
- obsługa plików (zdjęcia, MEDIA_ROOT),
- wykorzystanie Bootstrap do stylizacji.

Na podstawie dokumentacji tworzono własne rozwiązania dostosowane do projektu, a kod nie był bezpośrednio kopiowany.

Wykorzystanie sztucznej inteligencji:

Narzędzia AI były wykorzystywane w ograniczonym zakresie, głównie w sytuacjach wymagających dodatkowego wsparcia:

- rozwiązywanie problemów z importem danych CSV i kodowaniem znaków,
- implementacja automatycznego pobierania zdjęć,
- debugowanie błędów w kodzie,
- pomoc przy elementach wizualnych (np. ikony),
- wsparcie przy redagowaniu dokumentacji.

AI nie było wykorzystywane do tworzenia całych funkcjonalności, a jedynie jako narzędzie pomocnicze.

15.2 Wykorzystane źródła internetowe

- **Django Documentation – Models**
  Django Software Foundation
  `https://docs.djangoproject.com/en/5.0/topics/db/models/`
  data dostępu: 2026-03-09
- **Django Documentation – Views**
  Django Software Foundation
  `https://docs.djangoproject.com/en/5.0/topics/http/views/`
  data dostępu: 2026-03-10
- **Django Documentation – Forms**
  Django Software Foundation
  `https://docs.djangoproject.com/en/5.0/topics/forms/`
  data dostępu: 2026-03-11
- **Django Documentation – Authentication**
  Django Software Foundation
  `https://docs.djangoproject.com/en/5.0/topics/auth/`
  data dostępu: 2026-03-12

- **Bootstrap Documentation – Layout and Components**
  Bootstrap Team
  `https://getbootstrap.com/docs/5.0/layout/grid/`
  data dostępu: 2026-03-13

- **Python CSV Module Documentation**
  Python Software Foundation
  `https://docs.python.org/3/library/csv.html`
  data dostępu: 2026-03-14

- **Pillow Documentation**
  Alex Clark and Contributors
  `https://pillow.readthedocs.io/`
  data dostępu: 2026-03-15

- **duckduckgo_search (ddgs) Documentation**
  PyPI Contributors
  `https://pypi.org/project/duckduckgo-search/`
  data dostępu: 2026-03-16

15.3 Wykorzystanie sztucznej inteligencji – źródła

- **ChatGPT (model GPT-5.3)**
  OpenAI
  `https://chat.openai.com/`
  data dostępu: 2026-03-17

15.4 Wykorzystane biblioteki i frameworki

- Python 3.10 – język programowania,
- Django 5.x – framework webowy,
- SQLite – relacyjna baza danych,
- Bootstrap 5 – framework CSS,
- Pillow – biblioteka do obsługi obrazów,
- duckduckgo_search / ddgs – biblioteka do wyszukiwania obrazów.